



ANALYSIS OF ALGORITHMS

Weekly Lecture Schedule • Fall 2026

Bachelor of Science in Information Technology • *Design, Proof and Measurement*

HEC 2023 Computing Core, semester 7. Every bound in this course is proved, and then measured on a real machine.

CREDIT HOURS 3 (3-0): 3 lecture hours a week	DURATION 16 teaching weeks; midterm Week 8, project due Week 16	PREREQUISITE Data Structures (HEC 2023, p. 65)	CONTENTS 23 chapters in 6 parts; 23 measured experiments in C++
--	---	--	---

📅	📖 Lecture topics	🔧 Measured experiment	📋 Assessment
1	1 The Role of Algorithms – what an algorithm is; the unit-cost RAM model and the three things it ignores; why the question is asymptotic 2 Input Size, Input Nature, Three Cases – size as bit length; best, worst and average case; why an average needs a stated distribution	The ratio test; the crossover at $n \approx 65$	—
2	3 Asymptotic Notation – O , Ω , Θ , o and ω defined and proved from the definitions; the limit test; four common abuses	Fit a curve; exclude the exponents either side	—
3	4 Correctness: Loop Invariants – initialisation, maintenance, termination; partial against total correctness; the loop variant	A bug that is never wrong and never finishes	—
4	5 Recurrences I: Substitution and Trees – guess and prove by induction; strengthening the hypothesis; recursion trees; unbalanced splits 6 Recurrences II: The Master Theorem – all three cases, the regularity condition, and a recurrence that falls in the gap	Recursion trees counted; the theorem verified exactly	—
5	7 Brute Force, and When It Is Right – exhaustive search as baseline and oracle; maximum subarray in n^3 , n^2 and n 8 Divide and Conquer: Merge Sort – the three-step pattern; correctness from the merge invariant; tuning the cutoff	Where n^2 beats $n \lg n$; merge sort tuned	—
6	9 Quicksort and Randomisation – the worst case is already-sorted input; expected $\Theta(n \lg n)$ on every input; introsort	The adversarial input, and what randomising costs	Quiz 1 (Ch 1–6)
7	10 Lower Bounds and Linear-Time Sorting – the decision-tree $\Omega(n \lg n)$ proof; counting and radix sort, and what each one assumes	Counting and radix sort against <code>std::sort</code>	Assignment 1 set
8	11 Selection and Order Statistics – quickselect; why deleting one recursive call changes the master-theorem case; median of medians	Midterm, then quickselect against a full sort	Midterm (Ch 1–9)
9	12 Heaps and Search Trees, Analysed – why a heap’s height cannot fail and a search tree’s can; Build-Heap is $\Theta(n)$ 13 Hashing, and What $O(1)$ Promises – simple uniform hashing; load factor; the two ways the assumption fails	A hash table degrading under a poor hash	Assignment 1 due
10	14 Greedy Algorithms – the greedy-choice property; exchange arguments; activity selection; Huffman coding proved optimal	Huffman coding against the entropy bound	—
11	15 Dynamic Programming I: The Elements – optimal substructure and overlapping subproblems; memoisation against bottom-up 16 Dynamic Programming II: The Classics – LCS, edit distance, 0/1 knapsack; why $\Theta(nW)$ is pseudo-polynomial	Memoised against bottom-up, timed and counted	—
12	17 Amortised Analysis – aggregate, accounting and potential methods; why n appends to a doubling array cost $\Theta(n)$	Aggregate cost against worst-case cost	Quiz 2 (Ch 10–16)
13	18 Graphs: Representation and Sparsity – matrix against list; why every bound carries two parameters; BFS, DFS, topological order 19 Minimum Spanning Trees – the cut property proved; Kruskal and Prim; union-find, where amortised analysis pays off	The matrix/list crossover as density falls	Assignment 2 set Project proposal
14	20 Shortest Paths – Dijkstra with its correctness proof; Bellman–Ford and negative edges; Floyd–Warshall for all pairs	Dijkstra with and without a priority queue	—
15	21 String Matching – naive, Rabin–Karp and KMP; the prefix function; why KMP has no bad case at all 22 Complexity Classes: P, NP, NP-Complete – verifying against finding; reductions; what to do when a problem is NP-complete	Naive, Rabin–Karp and KMP on one corpus	Quiz 3 (Ch 17–22) Assignment 2 due
16	23 The Semester Project – implement, measure and defend; the ten rules of experimental method; the measurement harness	Implement, measure, defend	Project due; Final (Ch 1–22)

ASSESSMENT WEIGHTS

20% Sessional: quizzes, assignments, project	30% Mid-term, Week 8	50% Final-term, all chapters
--	--------------------------------	--

I Analysing an Algorithm
IV The Design Paradigms

II Recursion and Divide-and-Conquer
V Graph Algorithms

III Sorting, Selection and the Structures Beneath
VI Strings, Limits and Practice

Follows HEC 2023 (*Curriculum of BS Information Technology*, p. 65), whose course outline is word for word identical to HEC 2017 (p. 89). Three topics are taught that the outline does not name, each justified in the handout’s Course Specification: the comparison lower bound (Week 7), the first result in the course about a *problem* rather than an algorithm; minimum spanning trees (Week 13), absent from the outline text but required by CLO-7 by name; and amortised analysis (Week 12), without which Chapter 13’s hash-table claim and Chapter 19’s union-find cannot be stated honestly, and which CLO-3 requires for sequences of operations. The published reading list misspells Cormen as “Corman” and cites CLRS 3rd ed. (2009); Appendix E of the handout cites the 4th (2022). Chapter numbers refer to the handout, *Analysis of Algorithms — Design, Proof and Measurement* (242 pp., 23 chapters, 7 appendices); Appendix G of that handout is the source of record for this page. Every chapter carries one experiment whose figures were produced on the machine described in Appendix A and pasted into the text; the sources are in code/ and each is reproducible. Weights follow the department’s standard split: Sessional 20%, Mid-term 30%, Final-term 50%.